C SOCKET PROGRAMMING — TCP & UDP (EXACT CODE, COMMANDS & VIVA)

Fedora Linux • GCC • Computer Networks Practical Exam

1 Socket Programming Overview

Socket An **endpoint** used for communication between programs over a network (IP address + port).

TCP **Connection-oriented**, reliable, ordered **byte stream**.

UDP **Connectionless** datagrams, **no guarantee** of delivery or order.

Common practicals

Echo server · Chat · String reversal · Uppercase conversion · Date/time server · File transfer

2 TCP Client–Server Diagram

`SOCK_STREAM`



TCP server

`socket()` → `bind()` → `listen()` → `accept()` → `recv()/send()` → `close()`

TCP client

`socket()` → `configure server address` → `connect()` → `send()/recv()` → `close()`

accept() returns a **NEW connected socket** descriptor (`client_fd`). The original socket (`listen_fd`) stays the **listening** socket.

3 UDP Client–Server Diagram

`SOCK_DGRAM`



UDP server

`socket()` → `bind()` → `recvfrom()` → `sendto()` → `close()`

UDP client

`socket()` → `sendto()` → `recvfrom()` *(if reply expected)* → `close()`

Basic UDP needs **no listen() or accept()** (and no connect()): every datagram carries its own destination address.

4 Headers & Address Structure

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
```

stdio: printf, perror · stdlib: exit · string: strlen · unistd: close · sys/socket: socket API · netinet/in: sockaddr_in, htons · arpa/inet: inet_pton

Term	Meaning
`AF_INET`	IPv4 address family
`SOCK_STREAM`	TCP socket type
`SOCK_DGRAM`	UDP socket type
`INADDR_ANY`	All local IPv4 interfaces
`htons()`	Port number → network byte order
`inet_pton()`	Text IPv4 address → binary form
`127.0.0.1`	Loopback address (this computer)
`sockaddr_in`	IPv4 socket address structure

Address structure (server side)

```c
struct sockaddr_in addr = {0};
addr.sin_family = AF_INET;
addr.sin_port = htons(5000);
addr.sin_addr.s_addr = INADDR_ANY;
```

Client side: put the server IP in the same structure with

```c
inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);
```

Byte order: the network uses big-endian. `htons()` = host to network, **s**hort (16-bit port). Forgetting it sends the wrong port.

5 Complete TCP Server

`server.c`

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

int main(void) {
    int listen_fd, client_fd;
    struct sockaddr_in addr = {0};
    char buf[1024];

    listen_fd = socket(AF_INET, SOCK_STREAM, 0); // 1. TCP socket
    if (listen_fd < 0) { perror("socket"); exit(1); }

    addr.sin_family = AF_INET;
    addr.sin_port = htons(5000);
    addr.sin_addr.s_addr = INADDR_ANY;
    if (bind(listen_fd, (struct sockaddr *)&addr,
             sizeof(addr)) < 0) {           // 2. bind port 5000
        perror("bind"); exit(1);
    }
    if (listen(listen_fd, 5) < 0) {         // 3. listen
        perror("listen"); exit(1);
    }
    printf("Server listening on port 5000...\n");

    client_fd = accept(listen_fd, NULL, NULL); // 4. NEW socket
    if (client_fd < 0) { perror("accept"); exit(1); }

    ssize_t n = recv(client_fd, buf,
                     sizeof(buf) - 1, 0);   // 5. receive
    if (n < 0) { perror("recv"); exit(1); }
    buf[n] = '\0';                          // null-terminate
    printf("Received: %s\n", buf);

    if (send(client_fd, buf, n, 0) < 0)     // 6. echo back
        perror("send");

    close(client_fd);                       // 7. close both
    close(listen_fd);
    return 0;
}
```

6 Complete TCP Client

`client.c`

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

int main(void) {
    int sock;
    struct sockaddr_in server = {0};
    const char *msg = "Hello from client";
    char buf[1024];

    sock = socket(AF_INET, SOCK_STREAM, 0); // 1. TCP socket
    if (sock < 0) { perror("socket"); exit(1); }

    server.sin_family = AF_INET;            // 2. server address
    server.sin_port = htons(5000);
    if (inet_pton(AF_INET, "127.0.0.1",
                  &server.sin_addr) != 1) {
        fprintf(stderr, "Invalid address\n"); exit(1);
    }

    if (connect(sock, (struct sockaddr *)&server,
                sizeof(server)) < 0) {      // 3. connect
        perror("connect"); exit(1);
    }

    if (send(sock, msg, strlen(msg), 0) < 0) { // 4. send
        perror("send"); exit(1);
    }

    ssize_t n = recv(sock, buf, sizeof(buf) - 1, 0); // 5. get echo
    if (n < 0) { perror("recv"); exit(1); }
    buf[n] = '\0';
    printf("Echo: %s\n", buf);

    close(sock);                            // 6. close socket
    return 0;
}
```

C SOCKET PROGRAMMING — TCP & UDP (EXACT CODE, COMMANDS & VIVA)

Fedora Linux • GCC • Computer Networks Practical Exam

7 UDP Server — complete

udp_server.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

int main(void) {
    int sock;
    struct sockaddr_in server = {0}, client = {0};
    socklen_t len = sizeof(client);
    char buf[1024];

    sock = socket(AF_INET, SOCK_DGRAM, 0);  // 1. UDP socket
    if (sock < 0) { perror("socket"); exit(1); }

    server.sin_family = AF_INET;
    server.sin_port = htons(5001);
    server.sin_addr.s_addr = INADDR_ANY;
    if (bind(sock, (struct sockaddr *)&server,
             sizeof(server)) < 0) {         // 2. bind port 5001
        perror("bind"); exit(1);
    }
    printf("UDP server waiting on port 5001...\n");

    ssize_t n = recvfrom(sock, buf, sizeof(buf) - 1, 0,
                         (struct sockaddr *)&client,
                         &len);             // 3. gets sender
    if (n < 0) { perror("recvfrom"); exit(1); }
    buf[n] = '\0';
    printf("Received: %s\n", buf);

    if (sendto(sock, buf, n, 0, (struct sockaddr *)&client,
               len) < 0)                    // 4. reply
        perror("sendto");

    close(sock);                            // 5. close socket
    return 0;
}
```

7 UDP Client — complete

udp_client.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

int main(void) {
    int sock;
    struct sockaddr_in server = {0};
    socklen_t len = sizeof(server);
    const char *msg = "Hello from client";
    char buf[1024];

    sock = socket(AF_INET, SOCK_DGRAM, 0);  // 1. UDP socket
    if (sock < 0) { perror("socket"); exit(1); }

    server.sin_family = AF_INET;            // 2. server address
    server.sin_port = htons(5001);
    if (inet_pton(AF_INET, "127.0.0.1",
                  &server.sin_addr) != 1) {
        fprintf(stderr, "Invalid address\n"); exit(1);
    }

    if (sendto(sock, msg, strlen(msg), 0,
               (struct sockaddr *)&server,
               sizeof(server)) < 0) {       // 3. send
        perror("sendto"); exit(1);
    }

    ssize_t n = recvfrom(sock, buf, sizeof(buf) - 1, 0,
                         (struct sockaddr *)&server,
                         &len);             // 4. get reply
    if (n < 0) { perror("recvfrom"); exit(1); }
    buf[n] = '\0';
    printf("Echo: %s\n", buf);

    close(sock);                            // 5. close socket
    return 0;
}
```

8 Important Socket Functions

Function	Purpose (practical-exam wording)
`socket()`	Creates a socket; returns a descriptor (**-1** on error).
`bind()`	Attaches the socket to a local IP address and port.
`listen()`	Makes a TCP socket passive and sets the pending-connection queue.
`accept()`	Takes one queued connection; returns a **new** socket for that client.
`connect()`	TCP client: connects its socket to the server IP and port.
`send()`	Sends bytes on a connected socket; returns bytes sent.
`recv()`	Receives bytes; returns count, **0** = peer closed, **-1** = error.
`sendto()`	Sends one datagram to a given address (UDP).
`recvfrom()`	Receives one datagram and fills in the sender's address (UDP).
`close()`	Closes the descriptor and frees the socket.
`htons()`	Converts a 16-bit value (port) to network byte order.
`inet_pton()`	Converts text IPv4 to binary; returns **1** on success.

Typical calls (fd = socket descriptor)

```c
socket(AF_INET, SOCK_STREAM, 0);   // UDP: SOCK_DGRAM
bind(fd, (struct sockaddr *)&addr, sizeof(addr));
listen(fd, 5);
cfd = accept(fd, NULL, NULL);
connect(fd, (struct sockaddr *)&addr, sizeof(addr));
send(fd, buf, len, 0);
n = recv(fd, buf, sizeof(buf) - 1, 0);
sendto(fd, buf, len, 0,
       (struct sockaddr *)&addr, sizeof(addr));
n = recvfrom(fd, buf, sizeof(buf) - 1, 0,
             (struct sockaddr *)&from, &fromlen);
```

9 Fedora Compile & Run Commands

```
$ gcc server.c -o server
$ gcc client.c -o client

# Terminal 1
$ ./server

# Terminal 2
$ ./client
Echo: Hello from client
```

Start the server first, then run the client in a **second terminal**. `127.0.0.1` lets both programs run on the **same computer**.

UDP (port 5001) — same idea

```
$ gcc udp_server.c -o udp_server
$ gcc udp_client.c -o udp_client
$ ./udp_server        # Terminal 1
$ ./udp_client        # Terminal 2
```

10 Common Practical Variations

only the changed part

Task	What changes in the TCP programs	Code idea (only the changed part)
Echo server	Send back the original message unchanged.	`send(client_fd, buf, n, 0);`
Reverse string	Reverse the received characters, then send.	`for (i=0, j=n-1; i<j; i++, j--) { char t=buf[i]; buf[i]=buf[j]; buf[j]=t; }`
Uppercase	Convert each character before sending. Add `<ctype.h>`.	`for (i=0; i<n; i++) buf[i] = toupper((unsigned char)buf[i]);`
Chat	Loop: receive, print, read a reply, send. Stop on an exit word or `n <= 0`.	`while ((n = recv(fd,buf,sizeof buf-1,0)) > 0) { buf[n]='\0'; if (!strncmp(buf,"exit",4)) break; ... fgets(...); send(...); }`
Date/time server	Build a time string and send it instead of echoing. Add `<time.h>`.	`time_t t=time(NULL); strftime(out,sizeof out,"%Y-%m-%d %H:%M:%S",localtime(&t)); send(client_fd,out,strlen(out),0);`
File transfer	Send in chunks. Loop until **all** bytes are sent; receiver loops until `recv()` returns 0.	`while ((r=fread(b,1,sizeof b,fp))>0) { size_t s=0; while (s<r) { ssize_t k=send(fd,b+s,r-s,0); if (k<0) break; s+=k; } }`
Multiple clients	Put `accept()` in a loop; handle each client in a child process (or a thread).	`while (1) { cfd=accept(lfd,NULL,NULL); if (fork()==0) { close(lfd); /* serve cfd */ close(cfd); exit(0); } close(cfd); }`

C SOCKET PROGRAMMING — TCP & UDP (EXACT CODE, COMMANDS & VIVA)

Fedora Linux • GCC • Computer Networks Practical Exam

11 TCP vs UDP Comparison

Feature	TCP	UDP
Connection setup	Connection-oriented: `connect()` / `accept()` (3-way handshake)	Connectionless: just `sendto()`, no setup
Reliability	Reliable: acknowledgements and retransmission	No guarantee of delivery
Ordering	Bytes arrive in order	Datagrams may arrive out of order
Data model	Byte stream, **no message boundaries**	Separate datagrams, boundaries kept
Socket type	`SOCK_STREAM` (sheet port 5000)	`SOCK_DGRAM` (sheet port 5001)
Main functions	`socket bind listen accept connect send recv close`	`socket bind sendto recvfrom close`
Typical applications	Web (HTTP), e-mail, file transfer, SSH	DNS queries, VoIP, video streaming, online games

Memory aid: TCP is like a phone call (connect first, reliable, in order). UDP is like posting letters (send any time; they may be lost or arrive out of order).

12 Common Errors & Troubleshooting

Situation	What it means and what to do
`Address already in use`	Port still held (e.g. TIME_WAIT after a recent run) or used by another program. Try another port, or call `setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof opt)` with `int opt = 1;` **before** `bind()`.
`Connection refused`	Nothing is listening there. Start the server first; check IP and port.
`recv()` returns 0	The TCP peer closed its sending direction. Stop reading and `close()`.
Incorrect port setup	Always use `htons()`. Writing `sin_port = 5000` on x86 really selects port 34835.
Wrong descriptor	Use the descriptor returned by `accept()` for client communication, not the listening one.
TCP message boundaries	One `send()` is **not** guaranteed to match one `recv()`. Loop; use a delimiter or length.
Error handling	Check every return value (`< 0` means error) and print the reason with `perror("name")`.
Resource cleanup	Close every socket after use: `close(client_fd); close(listen_fd);`

13 Viva Questions & Short Answers

18 questions

1 What is socket programming?
Writing programs that exchange data over a network using the socket API, usually as a client and a server.

2 What is a socket?
An endpoint for communication between programs over a network, identified by an IP address and a port.

3 Differentiate TCP and UDP.
TCP: connection-oriented, reliable, ordered byte stream. UDP: connectionless datagrams, no guaranteed delivery or order.

4 What does socket() do?
Creates a new socket and returns its descriptor (-1 on error), e.g. `socket(AF_INET, SOCK_STREAM, 0)`.

5 What is the purpose of bind()?
Assigns a local IP address and port number to the socket so clients can reach the server.

6 Why are listen() and accept() used?
listen() makes the TCP socket passive with a connection queue; accept() takes one connection and returns a new socket for that client.

7 What does connect() do?
A TCP client uses it to connect its socket to the server's IP address and port.

8 Difference between send() and sendto()?
send() is used on a connected (TCP) socket; sendto() also gives the destination address with each datagram (UDP).

9 Why is htons() used?
It converts the port number from host byte order to network byte order (big-endian).

10 What is INADDR_ANY?
A wildcard address (0.0.0.0): the server accepts traffic on all local IPv4 interfaces.

11 What is the loopback address?
127.0.0.1 (localhost): traffic stays on the same computer, so client and server can be tested together.

12 What does recv() returning zero mean for TCP?
The peer has closed the connection (end of data). Stop reading and close the socket.

13 Why do we close a socket?
To release the descriptor and system resources and to end the connection.

14 Can UDP guarantee delivery?
No. UDP is best effort: datagrams can be lost, duplicated or reordered, so the application must cope.

15 Does one TCP send() always match one recv()?
No. TCP is a byte stream: one send() may need several recv() calls, or several sends may arrive in one recv().

16 Listening vs connected socket?
The listening socket only waits for new connections. The socket from accept() carries data with one client.

17 Which calls does basic UDP skip?
listen(), accept() and connect(). It uses bind() on the server, then sendto() and recvfrom().

18 SOCK_STREAM vs SOCK_DGRAM?
SOCK_STREAM gives a TCP byte stream; SOCK_DGRAM gives UDP datagrams.

14 Last-Minute Exam Checklist

1. Start the **server first**, client in a 2nd terminal.
2. Ports match: **TCP 5000**, **UDP 5001**; IP **127.0.0.1**.
3. Use `htons()` for the port, `inet_pton()` for the IP.
4. TCP server talks on the **accept()** socket, not the listening one.
5. Null-terminate: `buf[n] = '\0';` before printing.
6. Check returns, `perror()`, then `close()` every socket.